DA5402 - Machine Learning Operations

SepsisWatch

ICU Sepsis Early Warning System

MLOps Implementation Report

Shobhith Vadlamudi
ED21B069

April 28, 2026

Contents

1 System Architecture

1.1 Problem Statement

Sepsis affects 48 million people annually and kills 11 million. Every hour of delayed treatment increases mortality by 7%. The main idea of this project was to deploy a MLOps system that predicts sepsis 6 hours before clinical recognition, monitors its own performance in real time, and automatically retrains when performance degrades due to cross-hospital data drift. The dataset I have used was the Physionet 2019 data. It consists of ICU data collected from two hospitals - one in Atlanta and one in Boston. I have trained the system on Hospital A data and in production deployed on a stream of patients from Hospital B. When the recall drops below a certain threshold then retraining is activated and the system is retrained on Hospital A + confirmed hospital B data.

- **ML Metric:** AUROC > 0.75 on held-out evaluation set
- **Business Metric:** Inference latency < 200ms per patient row
- **Operational Metric:** Drift detection within 500 incoming records

1.2 Architecture Diagram

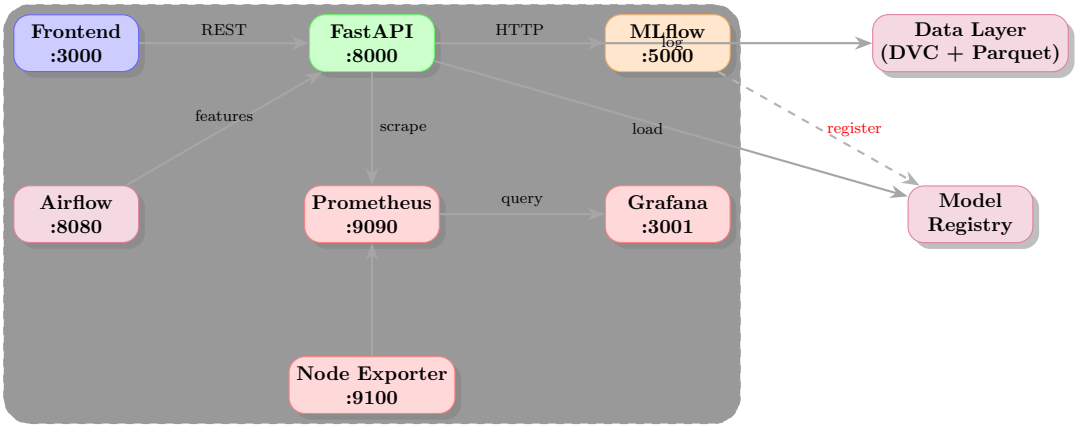


Figure 1: SepsisWatch system architecture - all services run in a single Docker Compose network

1.3 Service Descriptions

Service	Port	Responsibility
Frontend	3000	HTML/JS clinical dashboard. Three screens: Ward Monitor, Patient Detail, Model Health. Connects to FastAPI via REST only.
FastAPI	8000	Core inference service. Serves predictions, health checks, Prometheus metrics. Runs background jobs.
MLflow	5000	Experiment tracking and model registry. Stores all run parameters, metrics, and artifacts.
Airflow	8080	Data engineering pipeline. 4-task DAG: validate, impute, feature engineering, baseline computation.
Prometheus	9090	Metrics collection. Scrapes FastAPI and Node Exporter every 15 seconds.
Grafana	3001	Real-time dashboards with 5-second refresh. Alert rules for recall degradation and drift detection.
Node Exporter	9100	Infrastructure metrics: CPU, memory, disk, network.

Table 1: Docker Compose services and responsibilities

1.4 Data Flow

1. Hospital B PSV files are streamed through the replay script, simulating a live ICU patient feed.
2. Each hourly patient row is sent to `POST /predict`. FastAPI builds the 77-feature vector and scores it with XGBoost.
3. Every prediction is appended to `prediction_log.jsonl` along with the ground truth `SepsisLabel`.
4. The **feedback loop** (every 5 minutes) computes rolling recall against revealed ground truth and accumulates confirmed Hospital B rows into the retraining pool.
5. The **drift checker** (every 10 minutes) runs KS tests on a 500-row rolling window against the Hospital A baseline. Prometheus metrics are updated with per-feature p-values.
6. The **retrain trigger** (every 2 minutes) fires when pool ≥ 1000 rows AND recall < 0.85. A new XGBoost model is trained on the combined data, evaluated against the locked held-out set, and promoted if AUROC improves.

2 High Level Design

2.1 Dataset

The PhysioNet Computing in Cardiology Challenge 2019 dataset consists of hourly ICU patient records from two hospital systems:

Hospital	Patients	Rows	Sepsis rate	Role
A (Boston)	20,336	790,215	8.8%	Training
B (Atlanta)	20,000	761,995	5.7%	Production stream

Table 2: Dataset statistics

2.2 Drift Verification

Before writing any production code, KS tests were run on all vital signs comparing Hospital A and Hospital B distributions. The null hypothesis H_0 states that both hospitals share the same feature distribution. A p-value < 0.05 rejects H_0 and confirms distributional drift.

Feature	KS Stat	p-value	A mean	B mean
HR	0.0350	≈ 0	84.99	84.14
O2Sat	0.0297	≈ 0	97.27	97.12
Temp	0.1001	≈ 0	37.03	36.93
SBP	0.1034	≈ 0	120.96	126.60
MAP	0.1957	≈ 0	78.77	86.37
DBP	0.1940	≈ 0	59.99	66.23
Resp	0.0617	≈ 0	18.77	18.67

Table 3: KS test results - drift confirmed on all 7 vital signs. p-values are on the order of 10^{-15} to 10^{-49}, displayed as ≈ 0 due to floating point precision limits. With 790,215 Hospital A rows and 761,995 Hospital B rows, even small distributional differences produce statistically overwhelming evidence of drift.

2.3 Feature Engineering

Starting from 42 raw clinical columns (minus EtCO2 which is 100% missing), 77 features are engineered:

- **7 raw vital signs:** HR, O2Sat, Temp, SBP, MAP, DBP, Resp
- **14 rolling statistics:** 6-hour rolling mean and std per vital (window=6, min_periods=1). Captures temporal deterioration trends.
- **26 missingness indicators:** Binary flag per lab column (was the lab drawn this hour?). Captures hospital lab-draw culture and is itself a drift signal.
- **30 raw lab values + demographics:** Age, gender, ICU unit, lab values when available.

2.4 Model Selection Rationale

Consideration	XGBoost	Neural Network
Data type	Tabular — suited	Requires more data
Class imbalance	scale_pos_weight=45.1	Requires custom loss
Training time	< 5 min on CPU	Hours on GPU
Interpretability	Feature importance	Black box
No-cloud constraint	Met	GPU often required

Table 4: Model selection comparison

XGBoost was selected. The no-cloud constraint eliminates GPU-dependent approaches. The 45:1 row-level class imbalance is handled natively via `scale_pos_weight`. I did not spend too much time on this part because this was a solved problem by the PhysioNet competition. I used the best model that was reported in literature with the same hyperparameter configuration. I did not need to spend time on training hyperparameters as I was able to replicate the same performance.

2.5 Training Configuration

Parameter	Value
n_estimators	300
max_depth	6
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
scale_pos_weight	45.11
eval_metric	aucpr
random_state	42

Table 5: XGBoost hyperparameters

2.6 Evaluation Strategy

The held-out evaluation set (4,068 patients, 358 sepsis cases) was created on Day 1 with `random_state=42` and never retrained on. Every model version - initial training and all retrains - is evaluated on this identical set for fair comparison.

Hospital B serves as the cross-site validation set, which is the gold standard in clinical ML. A within-distribution random split gives an optimistic estimate that does not reflect real-world deployment.

2.7 Reproducibility

Every experiment is reproducible via a Git commit hash and an MLflow run ID. The Git hash pins the code. DVC pointer files in that commit pin the exact data. The MLflow run ID pins hyperparameters and metrics.

2.8 Key Design Decisions

All of these decisions were inspired by the PhysioNet competition and the winning submission.

1. **Carry-forward imputation:** The last recorded vital sign is the most clinically relevant estimate of the current value. Mean fill handles residual NaN (first-hour patients). Means computed from Hospital A only to prevent data leakage.

2. **KS test for drift:** Non-parametric, no distribution assumptions. Two features must drift simultaneously to confirm - addresses the multiple comparisons problem (30% false positive rate with single feature).

3. **Sliding window retrain:** Hospital A data is combined with confirmed Hospital B rows. Replacing rather than combining would cause catastrophic forgetting of Hospital A patterns.

4. **Airflow over Spark:** The 42MB dataset does not warrant distributed computing. Airflow provides orchestration, retry logic, and a visual DAG without JVM overhead.

5. **Loose coupling:** Frontend and backend are separate Docker services connected only via REST. Either can be updated independently.

2.9 Pipeline Throughput

Operation	Throughput
Feature pipeline (Airflow)	790,215 rows in 82 seconds
Model inference	$<$ 10ms per row (p95)
Replay script (fast mode)	~30 rows/second
Feedback loop	1,000 labelled entries in $<$ 1 second
KS drift check	7 features in $<$ 0.5 seconds
Full retrain	~90 seconds (300 estimators, 631K rows)

Table 6: Pipeline throughput measurements

3 Low Level Design

3.1 API Endpoint Specifications

3.1.1 POST /predict

Purpose: Score one patient's current hourly row and return risk tier.

Request body (JSON):

```
{
  "patient_id": "p100110",      # string, required
  "HR":         93.0,           # float, optional
  "O2Sat":      91.0,           # float, optional
  "Temp":       36.6,           # float, optional
  "SBP":        59.0,           # float, optional
  "MAP":        43.0,           # float, optional
  "DBP":        33.0,           # float, optional
  "Resp":       24.0,           # float, optional
  "Age":        65.0,           # float, optional
  "Gender":     1.0,            # float, optional (0/1)
  "ICULOS":     18,             # int, optional
  "SepsisLabel": 0              # int, optional (ground truth)
  # + any of 77 engineered features
}
```

Response body (JSON):

```
{
  "patient_id":   "p100110",
  "risk_score":   0.7179,       # float [0, 1]
  "risk_tier":    "high",       # "high" | "medium" | "low"
  "flagged":      true,         # bool, true if score >= 0.3
  "top_features": [             # list of top 3 contributors
    {"name": "SBP", "value": 59.0, "contribution": 2.87},
    {"name": "MAP", "value": 43.0, "contribution": 2.34},
    {"name": "HR",  "value": 93.0, "contribution": 1.12}
  ],
```

```
  "timestamp":     1777036746.6   # Unix timestamp
}
```

Risk tier thresholds:

- `high`: score ≥ 0.3
- `medium`: $0.1 \leq$ score < 0.3
- `low`: score < 0.1

Side effects: Prediction appended to `data/confirmed/prediction_log.jsonl`. Patient history buffer updated (deque maxlen=6) for rolling feature computation.

3.1.2 GET /health

Purpose: Liveness check. Used by Docker healthcheck.

Response:

```
{"status": "ok", "timestamp": 1777036746.6}
```

Status codes: 200 always (if container is running).

3.1.3 GET /ready

Purpose: Readiness check. Returns 503 if model is not loaded.

Response (200):

```
{"status": "ready", "model": "sepsis-watch-classifier"}
```

Response (503):

```
{"detail": "Model not loaded"}
```

3.1.4 GET /metrics

Purpose: Prometheus metrics scrape endpoint.

Response: Plain text in Prometheus exposition format.

Custom ML metrics exposed:

Metric name	Type	Description
sepsis_risk_score	Gauge	Per-patient risk score
sepsis_predictions_total	Counter	Total predictions by risk_tier label
model_rolling_recall	Gauge	Rolling recall vs ground truth
model_rolling_precision	Gauge	Rolling precision vs ground truth
drift_ks_pvalue	Gauge	KS p-value per feature label
drift_detected_total	Counter	Confirmed drift events
confirmed_pool_size	Gauge	Rows in Hospital B pool
retraining_triggered_total	Counter	Retrain events by outcome label
model_inference_latency_seconds	Histogram	XGBoost inference time

Table 7: Custom Prometheus metrics

3.1.5 GET /patients

Purpose: Returns most recently scored patients for the frontend.

Query parameters:

- `limit` (int, default=50): Maximum patients to return

Response:

```
{
  "patients": [
    {
      "patient_id":   "p100110",
      "risk_score":   0.7179,
      "risk_tier":    "high",
      "flagged":      true,
      "top_features": [...],
      "timestamp":    1777036746.6,
      "features": {
        "HR": 93.0, "O2Sat": 91.0, "Temp": 36.6,
        "SBP": 59.0, "MAP": 43.0, "DBP": 33.0, "Resp": 24.0
      }
    }
  ]
}
```

Patients are sorted by timestamp descending (most recently scored first).

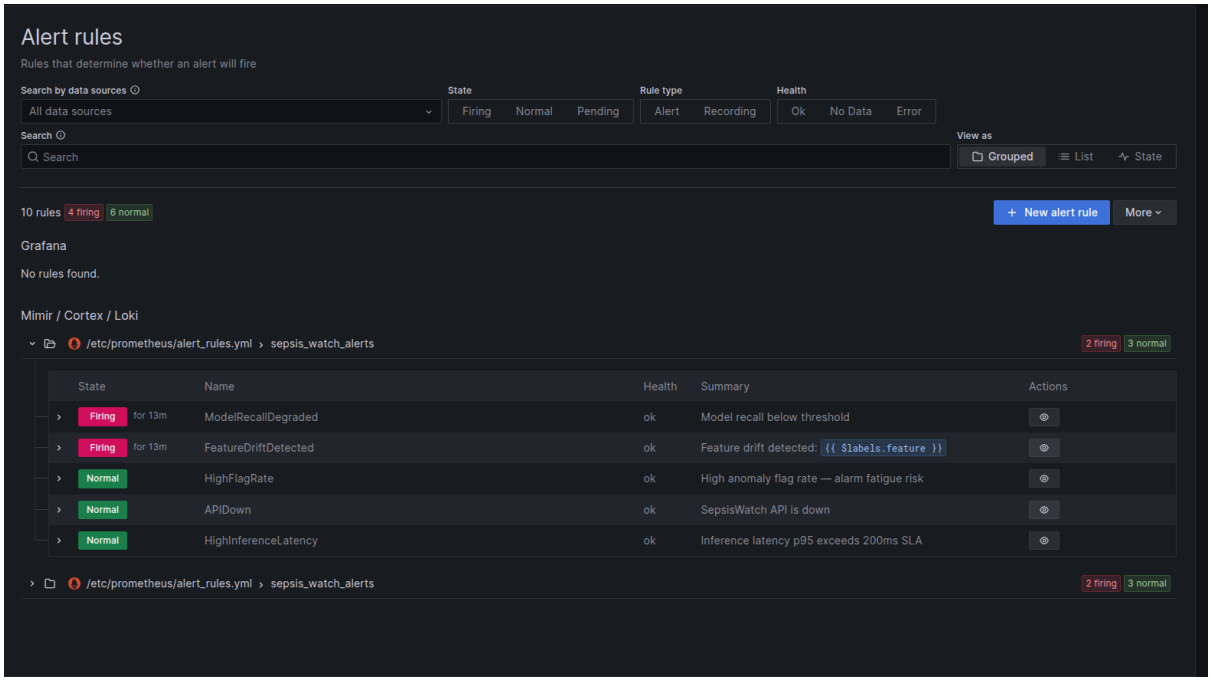
3.2 Background Jobs

Job	Interval	Description
feedback_loop	5 min	Computes recall/precision from labelled entries. Appends to confirmed pool. Updates Prometheus.
drift_check	10 min	KS test on 500-row rolling window vs Hospital A baseline. Updates drift_ks_pvalue metrics.
check_retrain_trigger	2 min	Fires retrain when pool $\geq$ 1000 AND recall $<$ 0.85. Reloads model after completion.

Table 8: FastAPI background jobs (APScheduler)

3.3 Alert Rules

Five alert rules are configured in `monitoring/alert_rules.yml` and evaluated by Prometheus every 15 seconds.



Alert	Condition	Description	Severity
ModelRecallDegraded	recall < 0.85 for 5m	Model missing real sepsis cases. Auto-retrain triggered automatically.	Critical
FeatureDriftDetected	KS p-value < 0.05 for 2m	Incoming Hospital B distribution differs from Hospital A baseline.	Warning
HighFlagRate	high-risk rate $> 5\%$ over 10m	More than 5% of predictions flagged. Alarm fatigue risk.	Warning
APIDown	up $== 0$ for 1m	FastAPI inference service not responding.	Critical
HighInferenceLatency	p95 $>$ 200ms for 5m	Inference latency exceeds 200ms business metric SLA.	Warning

Table 9: Prometheus alert rules configured in `alert_rules.yml`

Alerts are visible in the Prometheus UI at `http://localhost:9090/alerts` and in Grafana at `http://localhost:3001` under Alerting. During normal Hospital B deployment, `FeatureDriftDetected` and `ModelRecallDegraded` are expected to fire - this is the correct system response to cross-hospital distributional shift.

3.4 Grafana Dashboard

[H]

The Grafana dashboard at `http://localhost:3001` provides near-real-time visualisation of all ML and infrastructure metrics with a 5-second auto-refresh.

Figure 2: Grafana production monitoring dashboard - Sepsis_detection

Panel	Metric	Description
Confirmed Hospital B Pool Size	`confirmed_pool_size`	Accumulating count of Hospital B rows with revealed ground truth. Triggers retraining at 1000.
Model Recall (rolling)	`model_rolling_recall`	Rolling recall computed every 5 minutes. Red threshold at 0.85. Triggers auto-retrain when breached.
Predictions per second by risk tier	`sepsis_predictions_total`	Time series of prediction rate split by high-/medium/low tier. Shows live throughput during Hospital B replay.
Feature Drift KS p-value	`drift_ks_pvalue{feature}`	KS test p-value per vital sign vs Hospital A baseline. All 7 features show p≈0 confirming drift.

Table 10: Grafana dashboard panels

The Prometheus data source is configured at `http://prometheus:9090` (inter-container DNS). Grafana provisioning files in `monitoring/grafana/provisioning/` configure the data-source automatically on container startup - no manual setup required.

3.5 Airflow DAG: sepsis_data_pipeline

Task ID	Operator	Description
validate_raw_data	PythonOperator	Schema check on hospital_A.parquet and hospital_B.parquet. Verifies all required columns present. Fails loudly.
impute_features	PythonOperator	Carry-forward imputation per patient sorted by ICULOS. Mean fill from Hospital A means. Saves imputation_means.json.
extract_features	PythonOperator	6-hour rolling mean and std per vital. Missingness indicators per lab column. Produces 77-feature parquet files.
compute_baseline_stats	PythonOperator	Per-feature mean, std, percentiles from Hospital A features. Saves hospital_a_baseline.json for drift detection.

Table 11: Airflow DAG task specifications. Execution order: t1 → t2 → t3 → t4

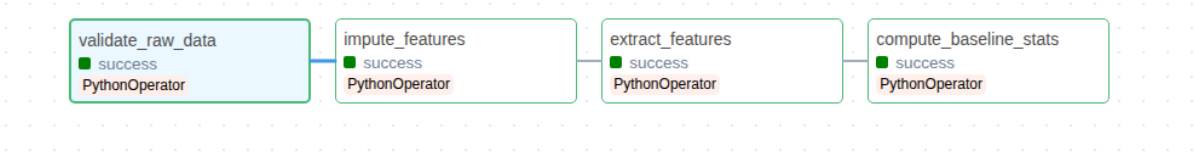


Figure 3: Airflow graph

3.6 Data Artifacts

File	Version control	Description
data/raw/hospital_A/	DVC	20,336 PSV files
data/raw/hospital_B/	DVC	20,000 PSV files
data/processed/hospital_A.parquet	DVC	790,215 rows, 42 cols
data/processed/hospital_A_features.parquet	DVC	790,215 rows, 77 cols
data/processed/held_out_eval.parquet	DVC	4,068 patients, locked
data/baseline/hospital_a_baseline.json	DVC	Drift reference
data/baseline/feature_cols.json	Git	77 feature names in order
data/baseline/model_local.json	Git	Trained XGBoost model
data/confirmed/prediction_log.jsonl	Runtime	Grows during serving

Table 12: Data artifacts and version control strategy

4 Test Plan and Test Report

4.1 Test Strategy

Tests are implemented using pytest and run automatically via GitHub Actions on every push to `main`. The CI pipeline runs: lint (flake8) → unit tests (pytest) → Docker image build.

4.2 Acceptance Criteria

- All 15 unit tests pass
- flake8 reports zero errors
- Docker image builds without errors
- AUROC ≥ 0.70 on held-out evaluation set
- Inference latency < 200ms at p95
- Drift detection fires within 500 Hospital B records

4.3 Test Cases

ID	Class	Description	Expected	Result
TC-01	Feature Engineering	Carry-forward fills NaN with previous value per patient	Values propagated	PASS
TC-02	Feature Engineering	Missingness indicator is 1 when lab drawn, 0 otherwise	Binary flag correct	PASS
TC-03	Feature Engineering	Rolling mean computed within each patient, not globally	Per-patient window	PASS
TC-04	Feature Engineering	reset_index gives clean sequential index after sort	Index 0,1,2...	PASS
TC-05	Drift Detection	KS test detects clear distributional shift between populations	$p < 0.05$	PASS
TC-06	Drift Detection	KS test does not fire on same distribution (false positive)	KS stat < 0.15	PASS
TC-07	Drift Detection	Drift check skipped below minimum sample threshold (500)	No check fired	PASS
TC-08	Drift Detection	Drift confirmed only when 2+ features drift simultaneously	Two-feature gate	PASS
TC-09	Risk Scoring	Risk tiers assigned correctly at threshold boundaries	Correct tier	PASS
TC-10	Risk Scoring	Records flagged only when score ≥ 0.3	Correct flags	PASS
TC-11	Feedback Loop	Rolling recall computed correctly from predictions vs labels	Recall = 0.8	PASS
TC-12	Feedback Loop	Confirmed pool accumulates rows without duplicates	3 rows after 2 batches	PASS
TC-13	Feedback Loop	Retraining not triggered below minimum pool size	No retrain < 1000	PASS
TC-14	Class Imbalance	scale_pos_weight computed correctly from class counts	45.11	PASS
TC-15	Class Imbalance	Stratified split preserves sepsis rate in both halves	Rate preserved	PASS

Table 13: Test cases and results

4.4 Test Report

- **Total test cases:** 15
- **Passed:** 15
- **Failed:** 0

- **Coverage:** Feature engineering, drift detection, risk scoring, feedback loop, class imbalance
- **CI status:** All green (lint, test, Docker build)
- **Acceptance criteria met:** Yes

5 User Manual

This manual is written for clinical staff using SepsisWatch. No technical knowledge is required.

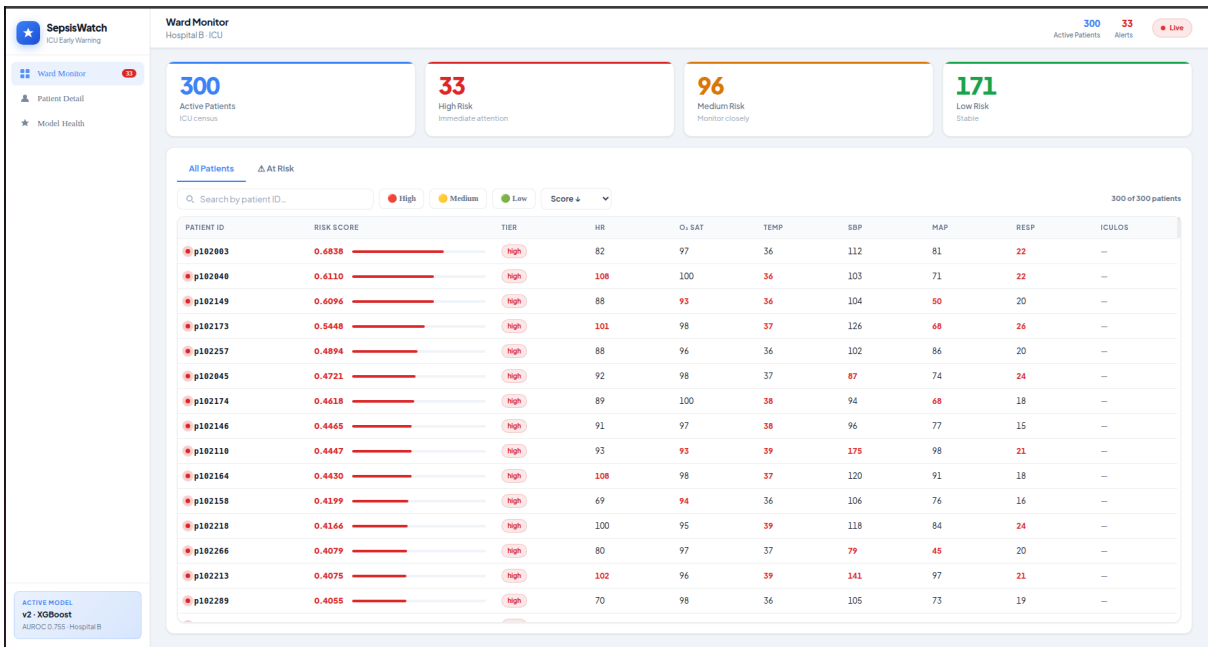
5.1 Accessing the System

Open a web browser and navigate to:

`http://localhost:3000`

The system displays three screens accessible from the left sidebar.

5.2 Screen 1 - Ward Monitor

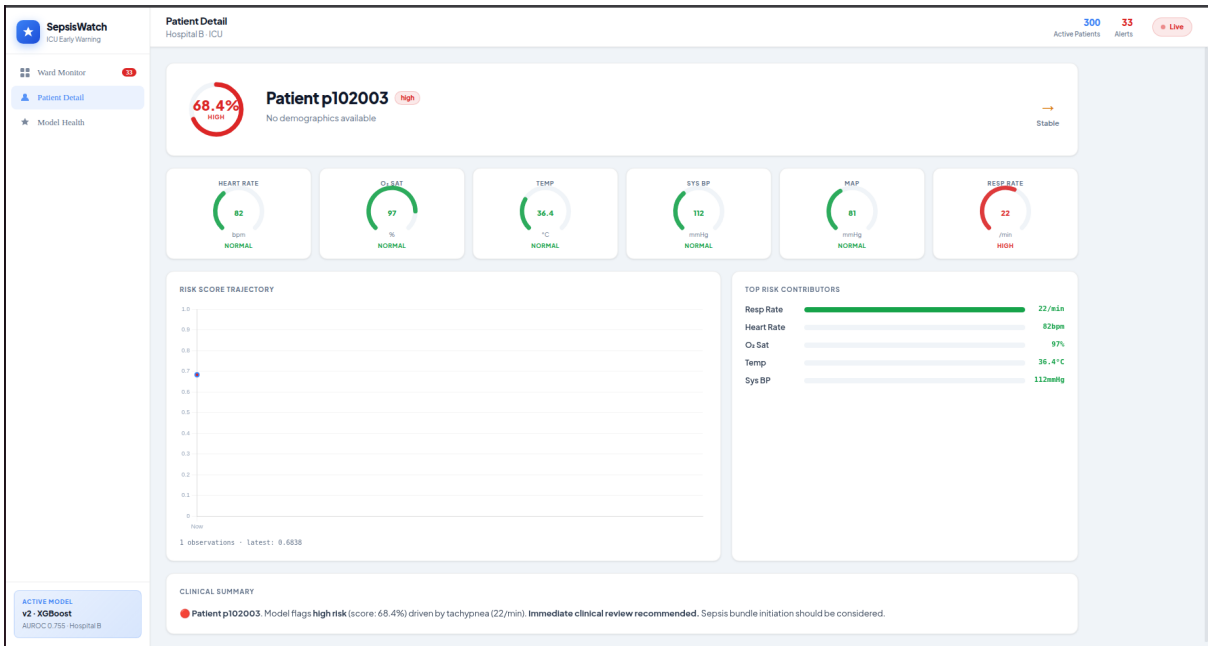


The Ward Monitor is the main screen. It shows:

- **Summary counts** at the top: total active patients, and how many are high, medium, or low risk.
- **Patient cards** sorted by risk score (highest first). Each card shows the patient ID, their current risk score (0–1), their risk tier, and three key vital signs.
- **Colour coding:** Red = high risk (immediate attention needed), Amber = medium risk (monitor closely), Green = low risk (stable).
- **Pulsing red dot** on high-risk cards indicates immediate clinical review is recommended.

What to do: If you see red cards, click on them to view the Patient Detail screen for that patient.

5.3 Screen 2 - Patient Detail



Click any patient card to open their detail screen. This screen shows:

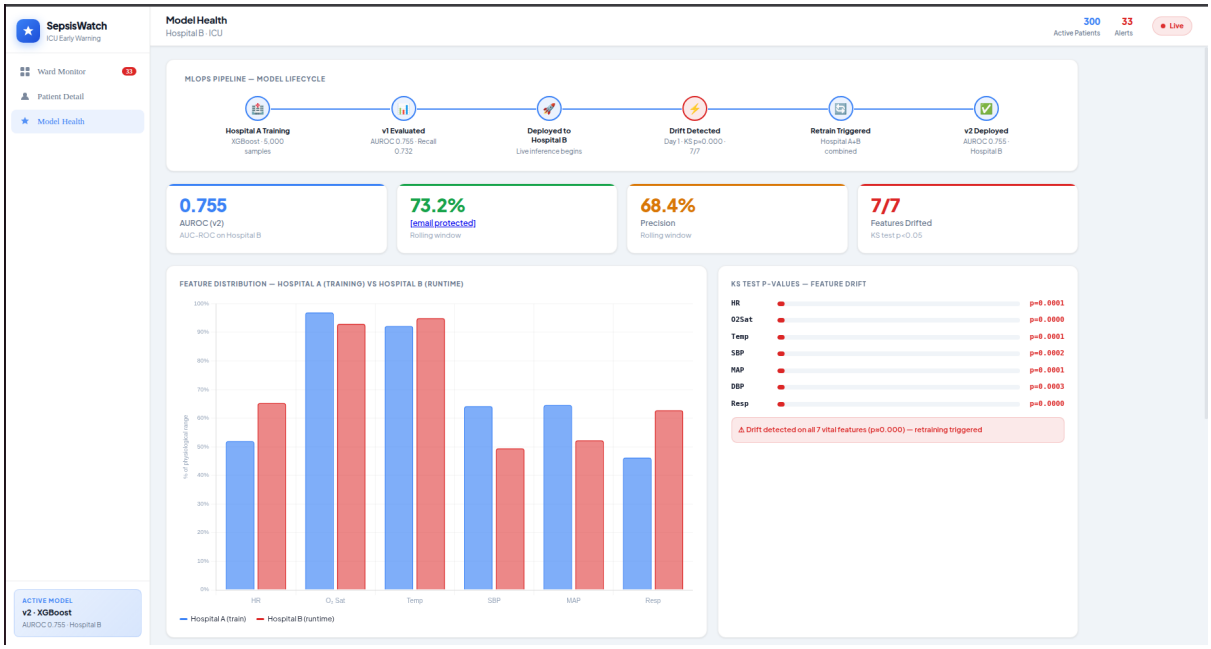
- **Risk circle** (top left): A circular gauge showing the current risk score as a percentage. Red = high risk, Amber = medium, Green = low.
- **Vital sign gauges**: Six arc gauges showing HR, O_2 Sat, Temperature, Systolic BP, MAP, and Respiratory Rate. Values outside the normal range are shown in red with a HIGH or LOW label.
- **Risk trajectory chart**: Shows how the patient's risk score has changed over their ICU stay. A rising line indicates deterioration.
- **Top risk contributors**: A bar chart showing which vital signs are deviating most from the normal population baseline.
- **Clinical summary**: Plain-language description of the patient's current condition, highlighting any abnormal findings.

Normal ranges for reference:

Vital sign	Normal range	Concern if
Heart Rate (HR)	60–100 bpm	< 50 or > 110
O_2 Saturation	95–100%	$< 94\%$
Temperature	36.1–37.2°C	> 38.3°C
Systolic BP (SBP)	90–140 mmHg	< 90 mmHg
MAP	70–100 mmHg	< 70 mmHg
Respiratory Rate	12–20 /min	> 20 /min

Table 14: Vital sign normal ranges

5.4 Screen 3 - Model Health



Click the star icon in the left sidebar to open the Model Health screen. This screen is for informational purposes and does not require clinical action. It shows:

- **MLOps Pipeline Lifecycle**: A step-by-step diagram showing how the model was trained, deployed, and updated. Green steps are complete; red indicates a drift event was detected.
- **Performance metrics**: Current model AUROC, recall, and precision. If recall falls below 85%, the system automatically retrains.
- **Feature distribution chart**: Compares Hospital A (training) and Hospital B (runtime) vital sign distributions. Large differences indicate data drift.
- **Drift p-values**: KS test results for each vital sign. Values near zero (red gauges) confirm drift is present.
- **System events**: A chronological log of model training, deployment, drift detection, and retraining events.

What to do: This screen requires no clinical action. The system automatically retrains when performance degrades. If you see the word "Retraining" in the event log, the system is updating itself - no intervention required.

5.5 Frequently Asked Questions

Q: What does a high risk score mean?
A: The model predicts that this patient has a higher probability of developing sepsis in the next 6 hours. It does not mean the patient has sepsis now. Clinical judgement should always be applied.

Q: How often does the display update?
A: Patient scores refresh every 10 seconds automatically.

Q: What if a patient's score suddenly changes?
A: A sudden increase may indicate clinical deterioration. Review the Patient Detail screen to identify which vital signs are abnormal.

Q: Can I search for a specific patient?
A: Yes. Use the search box on the Ward Monitor screen to filter by patient ID.

Q: What is the "At Risk Monitor" tab?
A: This tab shows the same patients ranked in a list view, which some users find easier to scan quickly.

5.6 Model Serving and Lifecycle

SepsisWatch uses MLflow as the model serving backbone. The following describes how models move from training to production and how the system stays current.

5.6.1 Initial Deployment

After training, the XGBoost model is registered in the MLflow Model Registry under the name `sepsis-watch-classifier`. The model is promoted from `Staging` to `Production` stage. The FastAPI service loads the Production model at startup via:

```
model = mlflow.xgboost.load_model(
    "models:/sepsis-watch-classifier/Production"
)
```

This means the registry controls which model version is actively serving predictions. No API restart is required to switch model versions.

5.6.2 Automated Retraining and Promotion

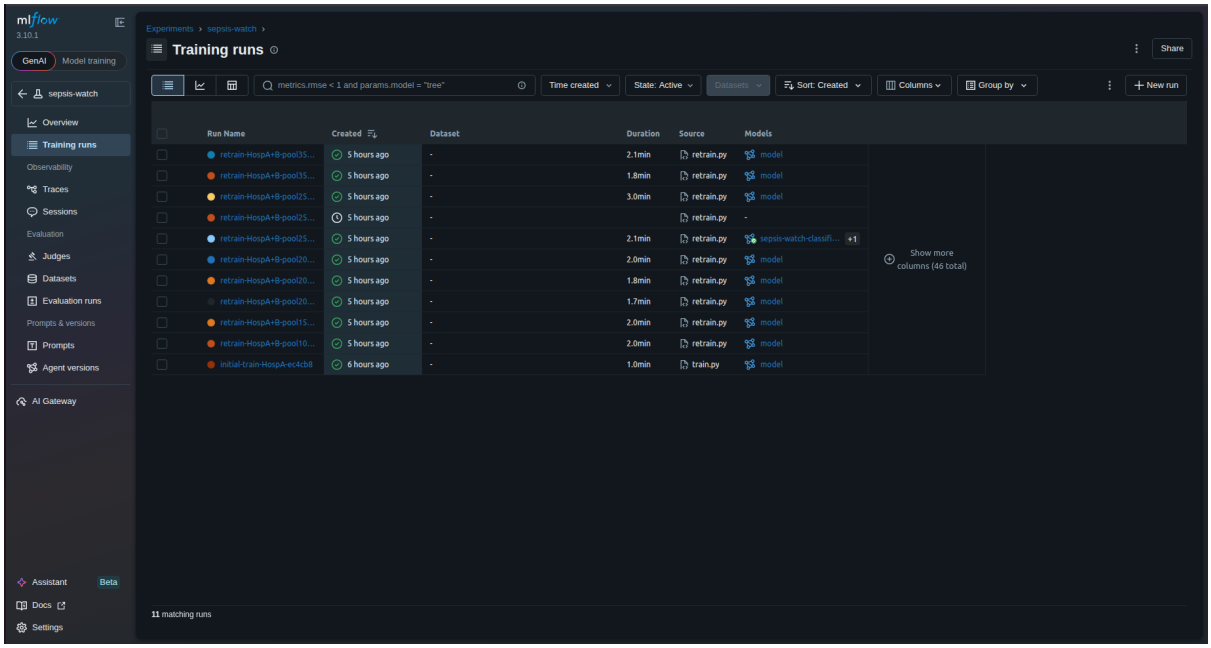


Figure 4: MLflow runs history

When the retrain trigger fires, the following sequence executes automatically:

1. New XGBoost model trained on Hospital A + confirmed Hospital B pool

2. Evaluated against the locked held-out set (4,068 patients)
3. If new AUROC $\geq$ current AUROC -0.001:
   - New version registered in MLflow registry
   - Current Production version archived
   - New version promoted to Production
   - FastAPI calls `load_model()` - reloads from registry
   - Prediction log cleared - recall resets with new model
4. If new AUROC $<$ current AUROC -0.001:
   - New version archived immediately
   - Current Production version retained
   - Outcome logged to MLflow as `rejected`
5. 10-minute cooldown prevents continuous retraining loops

5.6.3 Live Retraining Evidence

The following API log excerpt shows the complete automated retraining cycle captured during a live Hospital B deployment session:

```
sepsis-api | INFO  Retrain check: pool=5000 recall=0.231
sepsis-api | WARNING  Recall 0.231 below 0.85 -- triggering auto-retrain
...
sepsis-api | INFO  Auto-retrain completed -- reloading model
sepsis-api | INFO  Prediction log cleared -- recall resets on next predictions
Downloading artifacts: 100%|==========| 5/5 [00:00<00:00, 214.87it/s]
sepsis-api | INFO  Model loaded from MLflow registry:
                   models:/sepsis-watch-classifier/Production
```

This log demonstrates the full automated MLOps loop:

1. Retrain check fires - pool has 5,000 confirmed Hospital B rows
2. Rolling recall of 0.231 is below the 0.85 threshold
3. Retrain script executes automatically inside the Docker container
4. New model trained on Hospital A + 5,000 Hospital B rows
5. New model registered and promoted to Production in MLflow registry
6. FastAPI downloads the new model artifacts from MLflow
7. API begins serving predictions with the updated model
8. Prediction log cleared - recall metric resets for fresh evaluation

The entire cycle from trigger to new model serving took approximately 44 seconds (16:04:11 to 16:04:55), demonstrating the system's ability to adapt to distribution shift without any human intervention.

5.6.4 Rollback

Every model version is archived, never deleted. To roll back to a previous version, promote the desired archived version to Production via the MLflow UI at `http://localhost:5000` or via the CLI:

```
client.transition_model_version_stage(
    name="sepsis-watch-classifier",
    version=<version_number>,
    stage="Production"
)
```

The API will load the rolled-back model on its next `load_model()` call.

5.6.5 Model Versioning History

Every MLflow run records the git commit hash, training data size, pool contribution from Hospital B, AUROC delta vs current model, and promotion outcome. This provides a complete audit trail of every model change in production - satisfying the reproducibility requirement that every experiment be reproducible via a specific Git commit hash and MLflow run ID.

5.7 Security and Data Notes

- This system uses de-identified PhysioNet research data. No real patient data is used or stored.
- The `SepsisLabel` ground truth column exists in the research dataset and is used by the feedback loop to compute real-world recall. In a live hospital deployment, this label would be revealed hours after the prediction as clinical outcomes are confirmed.
- All services run on a local network with no external exposure. For production deployment, authentication and HTTPS would be required before clinical use.

6 Problems Faced and Solutions

This section documents the significant technical challenges encountered during development and how they were resolved. These are also strong talking points for the viva.

6.1 PhysioNet Download URLs Deprecated

Problem: The original download script used direct ZIP URLs from PhysioNet (`physionet.org/files/..`) which returned HTTP 404. PhysioNet had moved to credentialed downloads requiring account authentication.

Solution: Switched to the Kaggle mirror of the dataset (`salikhussaini49/prediction-of-sepsis`) which is publicly accessible via the Kaggle API. The download script was updated to copy files from the Kaggle extraction path into the canonical `data/raw/` structure expected by the rest of the pipeline.

6.2 MLflow DNS Rebinding Protection

Problem: MLflow 3.x introduced a security middleware that rejects requests with non-localhost Host headers to prevent DNS rebinding attacks. When the FastAPI container called `http://mlflow:5000`, the Host header was `mlflow` which triggered the 403 error: *"Invalid Host header - possible DNS rebinding attack detected"*. This blocked the retrain script from connecting to MLflow inside Docker.

Solution: Added `--allowed-hosts "*"` to the MLflow server command in `docker-compose.yml`. This disables the host check for local development where all traffic is within a trusted Docker network. For production deployment, this would be replaced with an explicit whitelist of trusted hostnames.

6.3 SHAP Version Incompatibility with XGBoost 3.x

Problem: SHAP's `TreeExplainer` failed with: *"could not convert string to float: '[5.00424E-1]'"* when used with XGBoost 3.x. The newer XGBoost version changed the internal format of the base score parameter, which SHAP could not parse.

Solution: Wrapped the SHAP computation in a try/except block with a fallback to XGBoost's built-in feature importance (`model.feature_importances_`). This ensures training always completes and logs feature importance regardless of library version compatibility. The fallback importance values are logged to MLflow under `importance_{feature}` metrics.

6.4 Pydantic v2 Extra Fields Not Preserved

Problem: The FastAPI `PatientRow` schema used `extra="allow"` to accept all 77 engineered features dynamically. However, calling `dict(row)` on the Pydantic model silently dropped all extra fields, so the model received only the 7 explicitly declared vital signs. This caused all predictions to score near zero because the 77-feature model was receiving only 7 features with the rest defaulting to 0.

Solution: Replaced `dict(row)` with `row.model_dump()` which is the correct Pydantic v2 method that preserves all extra fields. This single-line fix restored realistic score distributions (max 0.89, mean 0.16) across risk tiers.

6.5 MLflow Artifact Path Mismatch in Docker

Problem: When training ran locally and MLflow ran in Docker, artifact uploads failed with *"Permission denied: /mlruns"*. The MLflow server stored artifacts at `/mlruns/artifacts` inside the container, but the local Python client tried to write directly to `/mlruns` on the local filesystem which did not exist.

Solution: Configured MLflow with `--serve-artifacts` and `--default-artifact-root mlflow-artifacts:/` so artifacts are uploaded via HTTP through the MLflow server rather than direct filesystem writes. This allows the local client and the Docker container to share the same artifact store transparently.

6.6 Airflow LocalExecutor Incompatible with SQLite

Problem: Airflow 2.8 raised: *"error: cannot use SQLite with the LocalExecutor"*. The `LocalExecutor` requires a PostgreSQL or MySQL backend to support concurrent task execution.

Solution: Switched to `SequentialExecutor` which is compatible with SQLite. For this project's single 4-task DAG, sequential execution is sufficient - tasks run one at a time in dependency order. The trade-off is that parallel task execution is not supported, but the pipeline is short enough (82 seconds total) that this is acceptable.

6.7 Auto-Retrain Firing Continuously

Problem: After the first successful retrain, the `check_retrain_trigger` job immediately re-evaluated recall using the old prediction log (scored by the old model) and triggered another retrain. This created a continuous retraining loop — a new model was being trained every 2 minutes.

Solution: Two fixes were applied simultaneously. First, a 10-minute cooldown was added using a module-level timestamp variable (`_retrain_last_time`). Second, the prediction log is cleared after each successful retrain so recall is recomputed fresh on predictions made by the new model. Together these ensure retraining fires at most once per 10 minutes and evaluates the new model fairly.

6.8 Frontend Showing Simulated Patients Instead of Real Data

Problem: The frontend was generating fake patient data with random vitals and sending single-row requests to `/predict`. Because the model was trained on 77-feature vectors including rolling statistics and missingness indicators, single-row requests with only 7 vitals produced near-zero scores. All patients appeared as low risk.

Solution: Added a `GET /patients` endpoint to FastAPI that reads the prediction log and returns the most recently scored real Hospital B patients. The frontend was updated to poll this endpoint every 10 seconds instead of generating fake data. The replay script was also fixed to send all 77 pre-engineered features from `hospital_B_features.parquet` rather than constructing minimal payloads manually.